

# Optimality and Trade-offs in Fast BFT SMR (Extended Version)

Neil Giridharan ✉

University of California, Berkeley, CA, USA

Minh Tung Nguyen ✉

Institut Polytechnique de Paris, Palaiseau, France

Ittai Abraham ✉

a16z Crypto Research, New York City, NY, USA

Allen Clement ✉

Subzero Labs, Zurich, Switzerland

Natacha Crooks ✉

University of California, Berkeley, CA, USA

Pierre Sutra ✉

Institut Polytechnique de Paris, Palaiseau, France

## Abstract

Fast state-machine replication (SMR) protocols in the crash-fault setting have attracted significant interest in both academia and industry. This interest stems from their advantages over leader-based protocols, including low execution latency for non-conflicting commands, high throughput, improved availability, and increased fairness. While Fast SMR is well studied in the crash-fault setting, the Byzantine fault-tolerant (BFT) setting remains much less understood, with only a small number of existing Fast BFT protocols. In this paper, we present tight upper and lower bounds on the replication factor required for Fast BFT SMR. We also present a suboptimal protocol that illustrates a trade-off between replication factor and recovery efficiency.

**2012 ACM Subject Classification** Theory of computation → Distributed algorithms

**Keywords and phrases** Fast state machine replication, consensus, partial synchrony, fault-tolerance

**Digital Object Identifier** 10.4230/LIPIcs.DISC.2026.6

## 1 Introduction

State machine replication (SMR) is a foundational technique for building fault-tolerant services, ranging from replicated databases, coordination services to blockchains. SMR guarantees that all replicas execute the same sequence of commands such that they behave as a single highly available state machine.

Most widely deployed SMR protocols, including Raft [33] and Multi-Paxos [49], are *leader-based*. There is a distinguished replica that is responsible for assigning commands to positions in a global log. This design is simple and effective, but adds additional latency in two ways. First, every command must be routed through, and ordered by the leader. Second, in protocols such as Multi-Paxos, commands are assigned to predefined log slots. As a result, a command may be forced to wait for earlier slots to commit before it can execute, even when those earlier commands do not conflict with it.

Fast SMR protocols [32, 36, 15, 50] are designed to overcome the performance limitations inherent in traditional leader-based protocols. These protocols move away from a model where a single leader must totally order every command and instead exploit the observation that many commands commute. To ensure linearizability [23], the gold standard for SMR [39], replicas need only agree on a consistent order for conflicting commands; non-conflicting



© Neil Giridharan, Minh Tung Nguyen, Ittai Abraham, Allen Clement, Natacha Crooks, and Pierre Sutra;

licensed under Creative Commons License CC-BY 4.0

40th International Symposium on Distributed Computing (DISC 2026).

Editors: Ioannis Chatzigiannakis, Andrea Vitaletti, Keren Censor-Hillel, and William K. Moses Jr.; Article No. 6; pp. 6:1–6:21



Leibniz International Proceedings in Informatics

Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

commands can safely be executed in different orders [28]. Rather than forcing every command into a fixed slot within a totally ordered log, it is sufficient to reach agreement on the specific dependencies (ordering constraints) of each command.

This shift in perspective allows Fast SMR protocols to execute non-conflicting commands with low latency. Specifically, the defining property of Fast SMR protocols is that they can execute such commands in at most  $2\Delta$  time (two message delays) when the network is synchronous. Fast SMR protocols also provide three additional practical benefits. 1) High Throughput: By avoiding the single-leader bottleneck, the system spreads load across all replicas, maximizing aggregate bandwidth. 2) Improved Availability: Unlike leader-based systems that stall during a leader election, leaderless protocols continue to commit commands as long as the number of faults remains below the threshold  $f$ . 3) Increased Fairness: Because ordering decisions are decentralized, no single replica can order commands arbitrarily. This inherently mitigates the risks of biased ordering often seen in leader-based protocols.

Fast SMR protocols have gathered significant interest in both industry and academic settings. Protocols such as EPaxos [32], Atlas [15], Caesar [6], Accord [42], and BPaxos [51] confirm the substantial benefits that the approach enables, especially for workloads with low conflict rates. In fact, Accord is now being deployed in Apple’s Cassandra [5] system. Beyond these practical benefits, the crash-fault setting is also relatively well understood: prior academic work has formalized results for latency and resilience of these systems [36, 38, 37].

While crash-fault leaderless protocols have led to a substantial body of work and practical deployments, we lack a comparable understanding of what is achievable when replicas may behave Byzantine. In particular, it is not clear which benefits of Fast SMR can be preserved, what is the necessary replication factor, or what new mechanisms are required to maintain safety. The difficulty of the Byzantine setting is illustrated by prior attempts to adapt Fast SMR techniques to Byzantine faults, which were later found to have correctness issues [7, 41]. This gap motivates our study of Fast BFT SMR.

This paper initiates a systematic study of Fast BFT SMR protocols and identifies a fundamental trade-off between optimal resilience and recovery efficiency. We highlight three key results.

- Our main result is an upper bound showing that Fast BFT SMR is achievable with the optimal replication factor of  $n \geq 5f + 1$  replicas (Algorithm 2). Prior Fast BFT SMR protocols with a replication factor of  $n \geq 5f + 1$  either required a fault-free fast path [36, 47] or tightly synchronized clocks [31, 35]. In contrast, our protocol requires neither: it tolerates up to  $f$  failures while preserving fast path execution and does not require tightly synchronized clocks. The key technical ingredient is a new recovery procedure that safely recovers fast-path decisions while ensuring that conflicting commands are always executed in a consistent order. This allows the protocol to simultaneously achieve an optimal replication factor and fast execution latency for non-conflicting commands.
- We then prove a matching lower bound, showing that this replication factor is tight (Theorem 7). Specifically, we show that no Fast BFT SMR protocol exists for  $n \leq 5f$ . The lower bound follows from the requirement that Fast protocols execute non-conflicting commands in two message delays (including client communication).
- Finally, we present a simpler but suboptimal Fast BFT SMR protocol for  $n \geq 7f + 1$  replicas (Algorithm 3). This protocol illustrates the trade-off between replication factor and recovery efficiency: with larger quorums, recovery no longer requires our new mechanism and can instead rely on a much simpler procedure that is more efficient.

## 2 Model and Definitions

### 2.1 System Model

We consider a system of  $n$  replicas  $\Pi = \{1, \dots, n\}$ . Replicas communicate over a reliable, authenticated, point-to-point network. A replica is *correct* if it follows the protocol and is *Byzantine* otherwise. Byzantine replicas may behave arbitrarily: they may equivocate, omit messages, send malformed messages, or coordinate their behavior with other Byzantine replicas. As is standard, we assume that at most  $f$  replicas are Byzantine.

We consider a static adversary: the adversary chooses the faulty replicas before the execution begins. The adversary may coordinate Byzantine replicas and schedule message deliveries subject to the network assumptions below, but cannot break standard cryptographic primitives.

We adopt the partially synchronous model. There is a known message-delay bound  $\Delta$  and an unknown Global Stabilization Time, denoted GST, such that after GST every message sent between correct replicas is delivered within  $\Delta$  time. Before GST, messages may be delayed arbitrarily, but the network does not corrupt messages.

We assume standard digital signatures and a public-key infrastructure. We write  $\langle m \rangle_k$  to denote a message  $m$  signed by replica  $k$ . A message is *well-formed* if all signatures it contains are valid and the message satisfies the syntactic requirements of the protocol. Correct replicas ignore malformed messages.

### 2.2 Definitions

**BFT SMR.** In Byzantine fault-tolerant state-machine replication, clients **submit** commands to be executed by replicas, and replicas **execute** commands by applying them to the state machine [39]. Without loss of generality, we assume that commands are unique and have unique, collision-resistant identifiers.

More precisely, there is a public verification predicate  $\text{VerifyId}(id, x)$  that determines whether  $id$  is a valid identifier for command payload  $x$ . We also assume that identifiers are unforgeable, so only the client that submitted command payload  $x$  can produce an identifier  $id$  such that  $\text{VerifyId}(id, x)$  holds. We also assume collision resistance: for any command payload  $x$  and identifier  $id$  such that  $\text{VerifyId}(id, x)$  holds, no adversary can find a distinct command payload  $y \neq x$  such that  $\text{VerifyId}(id, y)$  also holds. A standard way of generating identifiers that satisfies this definition is to use a signed digest of the command payload.

A command is valid if it satisfies the public verification predicate and has a valid client signature. A command  $x$  conflicts with command  $y$  (we write  $x \bowtie y$ ) if the relative order of executing these two commands affects the final system state. Two commands that do not conflict commute ( $x \nbowtie y$ ). The **noop** command is a special command that does not conflict with any other command. Database systems, for instance, say that transactions conflict if both of them access the same key and one of the accesses is a write. As observed in prior work [28, 34, 36], BFT SMR only needs to order conflicting commands to satisfy linearizability. We provide the formal definition of BFT SMR below.

► **Definition 1** (BFT SMR). *A protocol implements Byzantine fault-tolerant state machine replication if it satisfies the following properties when there are at most  $f$  failures.*

**Safety.** *Correct replicas execute conflicting commands in the same order.*

**Liveness.** *In an execution with a finite number of submitted commands, if a correct replica executes command  $c$  or a correct client submits command  $c$ , then all correct replicas eventually execute  $c$ .*

**External validity.** *Every command executed by a correct replica was submitted by some client.*

Note that we have a stronger assumption on liveness that guarantees termination only when there are a finite number of commands. This property matches the liveness properties that Fast protocols like EPaxos [32, 37] provide.

**Fast BFT SMR.** The defining characteristic of Fast BFT SMR protocols is that they can execute commands in the minimal amount of latency (within  $2\Delta$ ) in *synchronous conflict-free* execution. Informally, a synchronous conflict-free execution is one in which it is after GST and there are no concurrently submitted conflicting commands. This stringent requirement is not easily met. Leader-based protocols [11, 52, 17] cannot satisfy it as routing client requests to the leader necessarily adds an additional message delay. Similarly, in protocols with a predefined slot order such as Mir-BFT [45] a command in slot  $i$  may have to wait for all lower-numbered slots to commit before it can execute, even when those slots committed non-conflicting commands. We formally define this performance constraint during conflict-free executions, adapting the definitions from [37, 38] to the Byzantine setting.

► **Definition 2** (Conflict-freedom). *A command  $c$  precedes a command  $c'$  in an execution if, at the time  $c'$  is submitted by a client, all correct replicas have already executed  $c$ . Commands  $c$  and  $c'$  are concurrent if neither precedes the other. A command  $c$  is conflict-free if it does not conflict with any concurrent command.*

► **Definition 3** (Optimal execution latency). *After GST, if a correct client submits a valid conflict-free command  $x$  at time  $t$ , then every correct replica executes  $x$  by time  $t + 2\Delta$ .*

► **Definition 4** (Fast BFT SMR). *A Fast BFT SMR protocol satisfies all properties of BFT SMR and optimal execution latency.*

### 3 Fast SMR Background

In this section, we give an overview of how the crash-fault tolerant (CFT) family of Fast SMR protocols works. We use a similar framework to construct our upper bound protocol for the BFT setting.

Because non-conflicting commands need not be ordered, Fast SMR protocols can run many single-shot agreement instances in parallel. Rather forming a totally ordered log of instances, the Fast SMR protocol constructs a partial order over instances (represented by command identifiers) such that any two conflicting commands need to be consistently ordered across replicas. This is achieved by having each instance agree on a tuple consisting of two components: 1) the command (payload) itself and 2) the dependencies of the command, which is the set of conflicting commands (specifically command identifiers) that this operation needs to be ordered after.

The collection of committed commands defines a dependency graph. Each committed command is a vertex, and there is an edge from each command to every command in its committed dependency set. Since replicas run an agreement instance for every command, all correct replicas agree on the dependency graph. Execution is then deterministic. A replica waits until all transitive dependencies of a command have committed, computes the strongly connected components of the resulting dependency graph, orders those components in topological order, and executes commands inside each component by increasing command identifiers. Because the linearization algorithm is deterministic, all correct replicas execute the same commands in the same order. The execution algorithm pseudocode is presented in Algorithm 1.

To maintain safety, Fast SMR protocols need to satisfy two key invariants [36, 37], which ensure that conflicting commands are always executed in the same order. The first invariant ensures that replicas agree on the same dependency graph. The second variant ensures that committed conflicting commands always have an edge in the dependency graph.

► **Theorem 5** (Invariant 1: Agreement). *For any command identifier  $id$ , if both  $(c, D)$  and  $(c', D')$  are committed for  $id$ , then  $c = c'$  and  $D = D'$ .*

► **Theorem 6** (Invariant 2: Visibility). *If  $(c, D)$  is committed and  $(c', D')$  is committed for command identifiers  $id$  and  $id'$  respectively, where  $id \neq id'$  and  $c \bowtie c'$ , then  $id \in D'$  or  $id' \in D$ .*

Fast SMR protocols consist of three components: a fast path that is guaranteed to succeed for conflict-free commands after GST. A recovery path, which guarantees that if the fast path fails, some value is eventually committed. Finally, an execution engine that executes commands

To achieve optimal execution latency, the agreement protocol includes a fast path that succeeds when a quorum of replicas have the same input value. In synchronous executions, conflict-free commands are guaranteed to take this fast path. Moreover, because all replicas have already executed the command's committed dependencies, the command can be executed immediately once it commits.

If the fast path does not succeed, either due to asynchrony or concurrent conflicting commands, the recovery path guarantees that some value is eventually committed. Recovery is view-based: each view has a coordinator responsible for driving agreement. To preserve safety, the coordinator must propose a value that is consistent with any value that may have committed on the fast path or in an earlier view, while also ensuring that conflicting commands cannot miss each other.

#### ■ Algorithm 1 Execution algorithm

---

```

1:  $executed \leftarrow \emptyset$ 
2: while true do
3:   Let  $G \subseteq dep$  be the largest subgraph such that  $\forall id \in G$ ,  $id$  is committed and all of
     its dependencies are in  $G$ 
4:   for  $C \in SCC(G)$  in topological order do
5:     for  $id \in C$  in increasing rank order of identifiers do
6:       if  $id \notin executed$  then
7:         execute command committed for  $id$ 
8:          $executed \leftarrow executed \cup \{id\}$ 

```

---

## 4 $n \geq 5f + 1$ Protocol Overview

We first show that there exists a protocol that solves Fast BFT SMR for  $n \geq 5f + 1$  replicas. The protocol consists of a series of instances, each identified by a request identifier  $id$ . A client request  $\langle \text{REQ}, id, x \rangle$  associates instance  $id$  with command payload  $x$ . The purpose of instance  $id$  is to commit a value  $(x', D)$ , where  $x'$  is either a command payload or **noop** and  $D$  is a set of dependencies that represent ordering constraints. Once a replica has committed all of the transitive dependencies for the instance, it can run the execution algorithm (Algorithm 1) to execute the command payload.

Each instance consists of two commit paths: (i) a fast path that commits in two message delays after GST when there are no concurrent conflicts and (ii) a recovery path that makes progress when conflicts do occur.

#### 4.1 Fast Path

To satisfy optimal execution latency, the protocol must commit conflict-free commands in two message delays after GST. Since the client already uses one message delay to send the command to replicas, the protocol has only one additional message delay before it must commit. The fast path therefore consists of replicas broadcasting a vote for their initial value. Since the fast path must tolerate up to  $f$  faulty replicas, a replica must commit after receiving  $n - f$  matching votes for the same value. If replicas observe non-matching votes, they enter the recovery path.

#### 4.2 Recovery Path

To guarantee liveness, the recovery path must also make progress using responses from only  $n - f$  replicas. At the same time, safety requires recovery to preserve any value that may already have been committed on the fast path.

If some replica committed a value from  $n - f$  fast votes, then by quorum intersection a later recovery quorum is guaranteed to observe only  $n - 3f$  of those votes. Thus, for safety, recovery must treat those  $n - 3f$  votes as sufficient evidence that a value may have been committed on the fast path. This is because up to  $f$  of the  $n - f$  votes may be from Byzantine replicas, while asynchrony may cause the  $n - f$  quorum to miss  $f$  correct replicas.

**Main challenge.** The main challenge for recovery comes from satisfying Invariant 6 (Visibility) when it is possible for replicas to receive  $n - 3f$  fast votes for  $(x, \emptyset)$  and  $n - 3f$  fast votes for  $(y, \emptyset)$  ( $x \bowtie y$ ), indicating that both could have gone fast path.

Consider for example two conflicting commands  $x$  and  $y$ , and two disjoint sets of replicas  $A$  and  $B$ , each of size  $n - 3f$ . Replicas in  $A$  receive  $x$  before  $y$  and send a fast vote for  $x$  with empty dependencies. Replicas in  $B$  receive  $y$  before  $x$  and send a fast vote for  $y$  with empty dependencies. Even though neither fast path actually succeeds, recovery for  $x$  may observe  $n - 3f$  fast votes for  $x$  with empty dependencies, while recovery for  $y$  may observe  $n - 3f$  fast votes for  $y$  with empty dependencies. Since each value appears consistent with a possible fast path commit, both recovery paths may try to commit their respective values. However, committing both values would violate Invariant 6 (Visibility):  $x$  and  $y$  would be committed with neither being in the dependency set of the other. The problem is that recovery cannot distinguish this execution from one in which the observed  $n - 3f$  votes are evidence of a real fast path commit.

**Key Idea.** Our key idea is to make recovery collaborative. While it may not be possible to determine whether a particular command committed on the fast path in isolation, it may be possible to do so after learning about the outcome of other commands! Our main contribution is a deadlock-free recovery path that waits to learn the outcomes of precisely those commands whose decisions could affect the command being recovered.

To make recovery deadlock-free, our protocol imposes a fixed total order on instance identifiers. Recovery is only allowed to wait downward in this order: a higher-ranked instance may wait for a lower-ranked instance, but a lower-ranked instance never waits for a higher-ranked one. This makes the wait-for graph acyclic. The lowest-ranked pending recovery cannot be blocked by another pending recovery, and once it completes, higher-ranked recoveries can continue.

- To determine which lower-ranked instances it must wait for, the recovery path begins with a validation phase. Replicas collect responses from  $n - f$  replicas, each reporting any fast vote it sent for a lower-ranked conflicting instance. If any response contains such a vote, recovery waits for that instance to commit before proceeding. Any lower-ranked instance that is not reported during the validation phase can be safely ignored. At most  $f$  correct replicas lie outside an  $n - f$  quorum, and at most  $f$  Byzantine replicas within the quorum could lie about their votes. Thus, at most  $2f$  replicas could have sent fast votes for a conflicting value, which is fewer than the  $n - 3f$  threshold required to recover a value.
- Recovery therefore has two rules. First, it may only wait for lower-ranked instances, which prevents deadlock. Second, it must wait for every conflicting lower-ranked instance discovered during the validation phase, which preserves safety. After those lower-ranked instances commit, recovery chooses a value that is consistent with their committed outcomes. If the value supported by its own fast-vote evidence is still safe, it recovers that value. Otherwise, it recovers **noop**, which is a special command that does not conflict with any other command.

## 5 $n \geq 5f + 1$ Protocol

Having sketched out an overview of the key techniques of the protocol, we now provide a detailed protocol specification for our Fast BFT state-machine replication protocol with  $n \geq 5f + 1$  replicas. The protocol is described in Algorithm 2. We assume that all messages are signed and that replicas verify both message signatures and message validity. For clarity, the pseudocode omits validation checks that ensure messages conform to the protocol specification.

**Fast path.** Clients issue requests to replicas by sending a  $\langle \text{REQ}, id, x \rangle$  message, where  $id$  is command identifier for a command payload  $x$ . Clients generate identifiers from taking a signed digest of  $(c_{id}, seq, x)$ , where  $c_{id}$  is a unique client id,  $seq$  is a monotonically increasing sequence number, and  $x$  is the command payload. When a replica receives a client request  $\langle \text{REQ}, id, x \rangle$ , either directly from the client or forwarded by another replica it first records the command  $x$  in *known* and forwards the request to all replicas if this is the first time it has seen the command (Algorithm 2, lines 9–10). If the replica has not already sent a fast vote or entered recovery (Algorithm 2, line 12), it computes the dependency set from conflicting commands in its local *known* state (Algorithm 2, line 13), records its fast vote (Algorithm 2, line 14), and broadcasts the FASTVOTE to all replicas (Algorithm 2, line 15).

A replica commits on the fast path after receiving at least  $n - f$  FASTVOTE messages with matching dependencies. It stores the resulting commit certificate, forwards the commit certificate to all replicas, and commits the value (Algorithm 2, lines 17–19). Importantly, replicas continue monitoring for fast votes and can commit on the fast path even if they have already entered the recovery path, and cannot send fast votes themselves. This is necessary for optimal execution latency: even in a synchronous conflict-free execution, a correct replica may enter recovery before it observes enough fast votes to commit on the fast path.

**Entering recovery.** If a replica receives  $n - f$  FASTVOTE messages with different dependencies, it stores this set of messages and forwards this evidence to other replicas (Algorithm 2, lines 22–21). Forwarding these fast votes ensures that all correct replicas eventually enter recovery. A replica waits for  $\Delta$  time before entering recovery (Algorithm 2, line 23). In synchronous conflict-free executions, this delay gives a client request enough time to reach the replica and prevents Byzantine replicas from forcing the replica into recovery before it has a chance to commit on the fast path. After this waiting period, replicas enter



recovery.

Recovery consists of a sequence of views, where each view has a dedicated coordinator, similar to classical leader-based BFT protocols. Each view consists of four phases: a status phase, a validation phase, a proposal phase, and a voting phase. The status phase determines whether some value might already have committed in a previous recovery view or on the fast path. The validation phase determines whether the coordinator must wait for lower-ranked conflicting instances before safely proposing a value. Once a safe value to propose has been identified, recovery runs standard proposal and voting phases, akin to traditional leader-based BFT protocols.

When entering any recovery view, a replica updates its view and stops replying to messages in earlier views (Algorithm 2, line 28). After this point, the replica sends no fast vote or messages from earlier views for that instance. The replica starts a view timer (Algorithm 2, line 29) and sends the coordinator a STATUS message containing the replica's fast vote and highest recovery vote for the instance (Algorithm 2, line 30).

**Status phase.** The coordinator of the view waits for  $n - f$  valid STATUS messages in the current view. If the collected STATUS messages contain  $n - 3f$  recovery votes for the same value, then that value might already have committed on the recovery path in an earlier view. Since these votes were sent after the validation phase, the coordinator proposes it directly (Algorithm 2, line 33). Otherwise, if the STATUS messages contain  $n - 3f$  FASTVOTE messages for the same value  $(x, \text{deps})$ , then that value might have committed on the fast path. In this case, the coordinator starts validation by broadcasting a VALIDATE message carrying the value and the status certificate (Algorithm 2, line 35). If neither condition holds, then the coordinator computes the union of the dependencies contained within the  $n - f$  FASTVOTE messages it used to enter view 1, and proposes this value (Algorithm 2, lines 38–39).

**Validation phase.** Validation is used only when the status phase finds a value that might have committed on the fast path. Upon receiving a valid VALIDATE message, a replica first checks that it is still in the indicated view (Algorithm 2, line 41). It then records the command  $x$  in *known* (Algorithm 2, line 43), ensuring that any future conflicting command learned by this replica must account for  $x$  in its dependency computation.

The replica next computes a set *wait* of lower-ranked instances that the coordinator must wait for before safely proposing  $(x, \text{deps})$  (Algorithm 2, line 44). For every lower-ranked instance  $id' < id$  for which the replica sent a FASTVOTE message, the replica checks whether the earlier command  $y$  for instance  $id'$ , conflicts with  $x$ , whether  $id$  is absent from the dependency set of  $y$ , and whether  $id'$  is absent from *deps* (Algorithm 2, lines 45–48). Violating a single one of these conditions is sufficient to guarantee that Invariant 6 holds: either the commands commute (precondition does not hold) or at least one command has a dependency on the other. If all of these conditions hold, then committing both values would violate Invariant 6, so the replica includes the corresponding client request in *wait*. The replica returns this information to the coordinator in a VALIDATEOK message (Algorithm 2, line 49).

The coordinator waits for  $n - f$  valid VALIDATEOK messages, unions their *wait* sets, and initializes the set of commit certificates it will collect (Algorithm 2, lines 51–52). For each lower-ranked request in the union, the coordinator forwards the request to all replicas so that the corresponding instance will eventually make progress (Algorithm 2, line 54). The coordinator then waits for a commit certificate for that instance (Algorithm 2, line 55), records the committed value and certificate (Algorithm 2, lines 56–57), and checks whether the committed value conflicts with the proposal in a way that violates safety (Algorithm 2, line 58). If such a conflicting commit is found, then the proposal could not have safely committed on



the fast path, and the coordinator proposes (`noop`,  $\emptyset$ ) instead (Algorithm 2, lines 59–60). If all waited-on instances commit values that do not violate safety, then the coordinator proposes the original value  $(x, \text{deps})$  together with the collected evidence (Algorithm 2, line 61).

**Proposal and voting.** A proposal contains the value for the current view together with enough evidence for replicas to verify that the coordinator followed the status and validation rules. A replica verifies that the coordinator picked the correct value to propose based on the given evidence (for clarity we omit this check from the pseudocode). When a replica receives a valid PROPOSE message, it ignores proposals for old views (Algorithm 2, lines 63–64) and refuses to vote twice in the same view (Algorithm 2, lines 65–66). If the proposed value is not `noop`, the replica records the command in *known* (Algorithm 2, lines 67–68). It then stores its recovery vote and broadcasts a VOTE message to all replicas (Algorithm 2, lines 69–70).

**Commit.** A recovery-path commit occurs when a replica receives  $n - f$  matching VOTE messages in the same view. The replica stores the resulting commit certificate, forwards it to all replicas, and commits the value (Algorithm 2, lines 72–74). When a replica commits either on the fast path or recovery path, it forwards all the REQ messages in the committed dependency set, so that eventually these dependencies will also be committed (Algorithm 2, lines 76–77).

**View change.** If progress stalls in a recovery view, the view timer expires and the replica broadcasts a VIEWCHANGE message (Algorithm 2, line 79). Once a replica receives  $f + 1$  such messages for a view, it enters the next view and forwards the view-change evidence to all replicas (Algorithm 2, lines 81–82). This mechanism ensures view synchronization among correct replicas.

**Execution.** When a replica commits a command and set of dependencies, it must wait until the set of transitive dependencies have been committed before its ready to execute. Execution then follows the same algorithm as in the EPaxos family of algorithms [32, 37], which is stated in Algorithm 1 for completeness.

**Resubmission.** A client may resubmit a command payload in case the previous instance committed `noop`, in which case the client sends a new request together with a commit certificate of the result of the previous instance. Replicas check the validity of this certificate before voting.

## 6 Lower Bound for Fast BFT SMR

We now show that the resilience bound  $n \geq 5f + 1$  is necessary. Thus, the protocol from the previous section is optimal with respect to resilience. The proof follows the structure of the FaB Paxos lower bound [3], but adapts the argument to the Fast BFT SMR setting. Throughout this section, we rely on the optimal execution latency, liveness, and safety properties stated in Section 2.

**Intuition.** The core difficulty is that, when  $n = 5f$ , correct replicas can be equally split on two different values. Consider an execution in any Fast BFT SMR protocol, in which  $2f$  correct replicas observe  $x$  before a conflicting command  $y$ , another  $2f$  correct replicas observe  $y$  before  $x$ , and the remaining  $f$  replicas are Byzantine. The Byzantine replicas can equivocate, making one correct replica see  $3f$  votes supporting  $x$  before  $y$ , while another correct replica sees  $3f$  votes supporting  $y$  before  $x$ .

Now focus on the replica that sees  $3f$  votes for  $x$  before  $y$ . From its perspective, this execution is indistinguishable from another execution in which  $3f$  correct replicas actually voted for  $x$  before  $y$ ,  $f$  correct replicas voted for  $y$  before  $x$ , and  $f$  replicas are Byzantine. In this execution, the Byzantine replicas can behave so that some correct replica observes  $4f$

---

**Algorithm 2** Replica  $i$  protocol for  $n \geq 5f + 1$ 


---

```

1:  $known \leftarrow \emptyset$  ▷ Set of received commands, used to compute dependency sets
2:  $fastvotes \leftarrow \emptyset$  ▷ Fast vote sent per instance
3:  $fv \leftarrow \emptyset$  ▷ Set of fast votes received per instance
4:  $votes \leftarrow \emptyset$  ▷ Highest vote (by view) sent
5:  $commits \leftarrow \emptyset$  ▷ Commit certificates
6:  $views \leftarrow \emptyset$  ▷ Current view for each instance

7: upon receiving  $r \leftarrow \langle \text{REQ}, id, x \rangle$  from client or replica do
8:   if  $r \notin known$  then
9:      $known \leftarrow known \cup \{r\}$ 
10:    forward  $\langle \text{REQ}, id, x \rangle$  to all
11:   if  $views[id] \geq 1$  or  $fastvotes[id] \neq \perp$  then
12:     return
13:    $deps \leftarrow \{(id', \langle \text{REQ}, id', y \rangle) \mid \exists \langle \text{REQ}, id', y \rangle \in known \text{ such that } y \bowtie x\}$ 
14:    $fastvotes[id] \leftarrow (x, deps)$ 
15:   send  $\langle \text{FASTVOTE}, id, (x, deps) \rangle$  to all

16: upon receiving  $C \leftarrow n - f$  matching  $\langle \text{FASTVOTE}, id, (x, deps) \rangle$  do ▷ Can fast commit in higher views
17:    $commits[id] \leftarrow C$ 
18:   forward  $C$  to all
19:   commit  $(id, (x, deps))$ 

20: upon receiving  $F \leftarrow n - f$  non-matching  $\langle \text{FASTVOTE}, id, \cdot \rangle$  do
21:    $fv[id] \leftarrow F$ 
22:   forward  $F$  to all
23:   wait  $\Delta$  ▷ Wait for possible fast path before entering view 1
24:   enter view 1 for instance  $id$ 

25: upon entering view  $v$  for instance  $id$  do
26:   if  $views[id] \neq \perp$  and  $views[id] \geq v$  then
27:     return
28:    $views[id] \leftarrow v$  ▷ After this point, replica  $i$  sends no fastvote/earlier view vote for  $id$ 
29:   start timer with duration  $6\Delta$  for view  $v$  in instance  $id$ 
30:   send  $\langle \text{STATUS}, id, v, fastvotes[id], votes[id] \rangle$  to  $\text{COORDINATOR}(id, v)$ 

31: upon receiving  $S \leftarrow n - f$  valid  $\langle \text{STATUS}, id, v, \cdot, \cdot \rangle$  and  $i = \text{COORDINATOR}(id, v)$  do
32:   if there exists  $n - 3f$   $votes[id]$  with  $val$  from  $S$  then
33:     send  $\langle \text{PROPOSE}, id, v, val, S, \perp, \perp \rangle$  to all ▷ Possibly went slow path
34:   else if there exists  $n - 3f$   $fastvotes[id]$  with  $(x, deps)$  from  $S$  then
35:     send  $\langle \text{VALIDATE}, id, v, (x, deps), S \rangle$  to all ▷ Possibly went fast path
36:   else
37:      $x \leftarrow$  unique command in  $fv[id]$ 
38:      $D \leftarrow$  union of all dependencies from  $fv[id]$ 
39:     send  $\langle \text{PROPOSE}, id, v, (x, D), S, \perp, \perp \rangle$  to all

```

---

---

**Algorithm 2** Replica  $i$  Protocol, continued
 

---

```

40: upon receiving valid  $\langle \text{VALIDATE}, id, v, (x, deps), S \rangle$  do
41:   if  $views[id] \neq v$  then
42:     exit upon
43:    $known \leftarrow known \cup \{ \langle \text{REQ}, id, x \rangle \}$ 
44:    $wait \leftarrow \{ \}$ 
45:   for  $\forall id' < id$  such that  $fastvotes[id'] \neq \perp$  do
46:      $(y, deps') \leftarrow fastvotes[id']$ 
47:     if  $y$  conflicts with  $x$  and  $id \notin deps'$  and  $id' \notin deps$  then
48:        $wait \leftarrow wait \cup \{ \langle \text{REQ}, id', y \rangle \}$  ▷ Client req message
49:   send  $\langle \text{VALIDATEOK}, id, v, (x, deps), S, wait \rangle$  to  $\text{COORDINATOR}(id, v)$ 

50: upon receiving  $V \leftarrow n - f$  valid  $\langle \text{VALIDATEOK}, id, v, (x, deps), S, wait \rangle$  and  $i = \text{COORDINATOR}(id, v)$  do
51:    $all\_waits \leftarrow$  union of all  $wait$  sets
52:    $commit\_certs \leftarrow \{ \}$ 
53:   for  $\forall \langle \text{REQ}, id', c' \rangle \in all\_waits$  do
54:     send  $\langle \text{REQ}, id', c' \rangle$  to all ▷ Ensures other correct replicas vote in  $id'$ 
55:     wait until  $commits[id'] \neq \perp$  before proceeding
56:      $val' \leftarrow$  committed value from  $commits[id']$ 
57:      $commit\_certs \leftarrow commit\_certs \cup \{ commits[id'] \}$ 
58:     if  $val' = (y, D')$  and  $y$  conflicts with  $x$  and  $id \notin D'$  and  $id' \notin deps$  then
59:       send  $\langle \text{PROPOSE}, id, v, noop, S, V, commits[id'] \rangle$  to all
60:     exit upon
61:   send  $\langle \text{PROPOSE}, id, v, (x, deps), S, V, commit\_certs \rangle$  to all ▷ no conflicting commit found

62: upon receiving valid  $\langle \text{PROPOSE}, id, v, val, \cdot, \cdot, \cdot \rangle$  do
63:   if  $views[id] \neq v$  then
64:     exit upon ▷ Do not vote in an old view
65:   if already voted for instance  $id$  in view  $v$  then
66:     exit upon
67:   if  $val = (x, deps)$  then
68:      $known \leftarrow known \cup \{ \langle \text{REQ}, id, x \rangle \}$ 
69:      $votes[id] \leftarrow (v, val)$ 
70:     send  $\langle \text{VOTE}, id, v, val \rangle$  to all

71: upon receiving  $C \leftarrow n - f$   $\langle \text{VOTE}, id, v, val \rangle$  do
72:    $commits[id] \leftarrow C$ 
73:   forward  $C$  to all
74:   commit  $(id, val)$ 

75: upon committing  $(id, (x, D))$  do
76:   for  $(id, msg) \in D$  do
77:     forward  $msg$  to all

78: upon view  $v$  timer for instance  $id$  expiring do
79:   send  $\langle \text{VIEWCHANGE}, id, v \rangle$  to all

80: upon receiving  $V \leftarrow f + 1$   $\langle \text{VIEWCHANGE}, id, v \rangle$  do
81:   enter view  $v + 1$  for instance  $id$ 
82:   forward  $V$  to all

```

---

votes for  $x$  before  $y$ . Since  $n - f = 4f$ , this is indistinguishable from an execution in which all correct replicas vote  $x$  before  $y$ . Therefore, by the latency and liveness requirements, the protocol can be forced to commit  $x$  before  $y$ .

By symmetry, another correct replica can be forced to commit  $y$  before  $x$ . Thus, correct replicas may execute conflicting commands in different orders, violating generalized Fast BFT SMR safety.

**Overview.** Our lower bound uses an indistinguishability argument to reach a contradiction. We split replicas into five disjoint groups each of size at most  $f$  ( $A$ ,  $B$ ,  $C$ ,  $D$ , and  $E$ ), and construct five executions  $(\sigma_1, \dots, \sigma_5)$ .

The construction proceeds in three stages. First,  $\sigma_1$  and  $\sigma_5$  are conflict-free synchronous executions for  $x$  and  $y$ , respectively. By the optimal execution latency property, the correct replicas in  $A$  execute  $x$  by time  $2\Delta$  in  $\sigma_1$  and execute  $y$  by time  $2\Delta$  in  $\sigma_5$ . In  $\sigma_1$  and  $\sigma_5$ , respectively, the replicas  $E$  and  $D$  are Byzantine and act crashed from the second round.

Second, we construct  $\sigma_3$ , the execution in which the contradiction arises. Both commands are submitted, the replicas in  $A$  are Byzantine and act crashed after the first round, and the adversary splits the remaining replicas: those in  $B \cup D$  receive  $x$  at time  $\Delta$  and  $y$  at time  $2\Delta$ , those in  $C \cup E$  the converse. During the first two rounds the former therefore behave exactly as they do in  $\sigma_1$ , and the latter exactly as they do in  $\sigma_5$ . Since both clients are correct and  $|A| \leq f$ , liveness forces every correct replica to eventually execute both commands; let  $R$  and  $S$  be the rounds at which  $b \in B$  and  $c \in C$  execute their first command.

Third, we show that  $b$  executes  $x$  before  $y$  in  $\sigma_3$  while  $c$  executes  $y$  before  $x$ , contradicting safety. For  $b$ , we build an execution  $\sigma_2$  in which the replicas in  $C$  are Byzantine and equivocate: toward  $A$  they behave as they do in  $\sigma_1$ , and toward the other replicas as they do in  $\sigma_3$ . Replicas in  $A$  thus cannot distinguish  $\sigma_2$  from  $\sigma_1$  through time  $2\Delta$  and executes  $x$  by that time, while external validity prevents it from having executed  $y$ ; liveness and safety then force  $b$  to execute  $x$  before  $y$  in  $\sigma_2$ . Since  $b$  cannot distinguish  $\sigma_2$  from  $\sigma_3$  through round  $R$ , the same holds in  $\sigma_3$ . The execution  $\sigma_4$  treats  $c$  symmetrically, with the replicas in  $B$  Byzantine and  $\sigma_5$  in place of  $\sigma_1$ .

**Preliminaries.** For some execution, we say that events that happen at the replicas during the time interval  $[0, \Delta)$  form *the first round*, events that happen during the time interval  $[\Delta, 2\Delta)$  *the second round*, and so on. Given a set of replicas  $R$  and an execution  $\sigma$ ,  $\mathbf{K}_\sigma^R$  denotes the steps taken by  $R$  in the  $K$ -th round of  $\sigma$ . Steps taken by Byzantine replicas are in **red**. An execution is *timely* [38, 27] when the faulty replicas behave correctly until time  $\Delta$ , after which they act crashed, the messages sent in a round are delivered precisely at the beginning of the next round ( $GST = 0$ ), and all local computations are instantaneous. In a timely execution, because the protocol is deterministic, the steps of replicas  $R$  during the first round are always the same; they are written  $\mathbf{1}^R$ .

► **Theorem 7** (Fast BFT lower bound). *Any deterministic Fast BFT SMR protocol requires  $n \geq 5f + 1$  replicas.*

**Proof.** Assume, for contradiction, that there exists a deterministic Fast BFT SMR protocol satisfying the stated properties with  $n \leq 5f$  replicas. Let  $\mathcal{E}$  be its set of executions. Fix two valid conflicting commands  $x$  and  $y$ . Since  $3f + 1 \leq n \leq 5f$ , partition the replicas into disjoint sets  $A$ ,  $B$ ,  $C$ ,  $D$ ,  $E$  such that  $|A|, |B|, |C|, |D|, |E| \leq f$ . Fix replicas  $a \in A$ ,  $b \in B$ , and  $c \in C$ . We construct five executions in  $\mathcal{E}$ , denoted  $\sigma_1, \sigma_2, \dots, \sigma_5$ . Executions  $\sigma_1$  and  $\sigma_5$  are timely.

Since the protocol is deterministic, identical timed local histories imply identical behavior.

**World 1** ( $\sigma_1$ ). This execution is timely. Replicas in  $E$  are Byzantine, and behave correctly until time  $\Delta$ , after which they act crashed. A correct client  $cl_x$  submits the valid command  $x$

at time 0 and no other command is submitted. Every replica in  $A \cup B \cup C \cup D$  receives  $x$  at exactly time  $\Delta$  (since the execution is timely). Messages among all replicas in the first round are all received at the beginning of the second round. Since  $|E| \leq f$ , this is a post-GST execution with at most  $f$  Byzantine replicas. Therefore, by optimal execution latency, every correct replica in  $A \cup B \cup C \cup D$  executes  $x$  by time  $2\Delta$ . In particular, replica  $a$  executes  $x$  by time  $2\Delta$ :

$$\text{submit}(x) \cdot \mathbf{1}^{A \cup B \cup C \cup D} \cdot \mathbf{1}^E \cdot \mathbf{2}_{\sigma_1}^{A \cup B \cup C \cup D} \cdot \text{execute}_a(x) \in \mathcal{E}$$

**World 5 ( $\sigma_5$ ).** This execution is timely and symmetric to  $\sigma_1$ . Replicas in  $D$  are Byzantine, and behave correctly until time  $\Delta$ , after which they act crashed. A correct client  $cl_y \neq cl_x$  submits the valid command  $y$  at time 0 and no other command is submitted. Every replica in  $A \cup B \cup C \cup E$  receives  $y$  at exactly time  $\Delta$ . Messages among all replicas in the first round are received at the beginning of the second round. Since  $|D| \leq f$ , this is a post-GST execution with at most  $f$  Byzantine replicas. Therefore, by optimal execution latency, every correct replica in  $A \cup B \cup C \cup E$  executes  $y$  by time  $2\Delta$ . In particular, replica  $a$  executes  $y$  by time  $2\Delta$ :

$$\text{submit}(y) \cdot \mathbf{1}^{A \cup B \cup C \cup E} \cdot \mathbf{1}^D \cdot \mathbf{2}_{\sigma_5}^{A \cup B \cup C \cup E} \cdot \text{execute}_a(y) \in \mathcal{E}$$

**World 3 ( $\sigma_3$ ).** Replicas in  $A$  are Byzantine, and behave correctly until time  $\Delta$ , after which they act crashed. Unless stated otherwise, messages are scheduled so that when they are sent in one round, they are delivered at the beginning of the next round. All local computations are instantaneous. Commands  $x$  and  $y$  are submitted by the same two correct clients as in  $\sigma_1$  and  $\sigma_5$ .

The adversary delays messages from clients so that replicas in  $B \cup D$  receive  $x$  at time  $\Delta$  and  $y$  at time  $2\Delta$ . Conversely, replicas in  $C \cup E$  receive  $y$  at time  $\Delta$  and  $x$  at time  $2\Delta$ . From what precedes, these two sets of replicas take the exact same first steps in  $\sigma_3$  as the ones they do during the first two rounds of  $\sigma_1$  and  $\sigma_5$ , respectively:

$$\sigma'_3 = \text{submit}(x) \cdot \text{submit}(y) \cdot \mathbf{1}^{B \cup C \cup D \cup E} \cdot \mathbf{1}^A \cdot \mathbf{2}_{\sigma_1}^{B \cup D} \cdot \mathbf{2}_{\sigma_5}^{C \cup E} \in \mathcal{E}$$

The two clients that submitted commands  $x$  and  $y$  are correct and only replicas in  $A$  are Byzantine, with  $|A| \leq f$ . Hence any continuation from  $\sigma'_3$  must uphold the properties of Fast BFT SMR. In particular, by liveness, all correct replicas eventually execute both  $x$  and  $y$ .

Let  $R$  (respectively  $S$ ) be the first round at which replica  $b$  (resp.,  $c$ ) executes a command, either  $x$  or  $y$ . Without loss of generality, suppose  $R \leq S$ :

$$\sigma'_3 \dots \mathbf{R}_{\sigma_3}^{B \cup C \cup D \cup E} \dots \mathbf{S}_{\sigma_3}^{B \cup C \cup D \cup E} \in \mathcal{E}$$

We will show that, in execution  $\sigma_3$ , replica  $b$  must execute  $x$  before  $y$ , while replica  $c$  must execute  $y$  before  $x$ .

**World 2 ( $\sigma_2$ ).** In  $\sigma_2$ , replicas in  $C$  are Byzantine. Let  $\text{GST} > \max\{R\Delta, 2\Delta\}$ . Thus, until time  $\max\{R\Delta, 2\Delta\}$ , the adversary may delay messages arbitrarily. Both  $x$  and  $y$  are submitted by clients. These two commands are received during the first round by  $C$ . The adversary delays messages from the clients so that replicas in  $A \cup B \cup D$  receive  $x$  exactly at time  $\Delta$ , and replicas in  $E$  receive  $y$  exactly at time  $\Delta$ . The adversary delays any message containing or referring to  $y$  to replica  $a$  until after time  $2\Delta$ . In the first round, replicas in  $A \cup B \cup D$  (respectively,  $E$ ) take the exact same steps as they do during the first round of  $\sigma_1$  (resp.,  $\sigma_5$ ).

$$\sigma'_2 = \text{submit}(x) \cdot \text{submit}(y) \cdot \mathbf{1}^{A \cup B \cup D} \cdot \mathbf{1}^E \in \mathcal{E}$$

The Byzantine replicas in  $C$  equivocate as follows. Toward replicas in  $A$ , they behave exactly as the correct replicas in  $C$  behave in  $\sigma_1$  through time  $2\Delta$ ; in particular, they act

as if they received  $x$  at exactly time  $\Delta$ . Toward replicas outside  $A$ , they behave exactly as the correct replicas in  $C$  behave in  $\sigma_3$  through time  $R\Delta$ ; in particular, they act as if they received  $y$  at exactly time  $\Delta$ .

Messages from replicas in  $A$  to replicas outside  $A$  that are sent in later rounds than the first one are delayed until after GST. The remaining messages are scheduled so that replica  $a$ 's timed local history through time  $2\Delta$  is identical as in  $\sigma_1$ , and replica  $b$ 's timed local history through time  $R\Delta$  is identical as in  $\sigma_3$ .

The first indistinguishability holds because, through time  $2\Delta$ , replica  $a$  receives first round messages from all replicas and  $x$  at exactly time  $\Delta$ , receives the same second round messages from replicas in  $A \cup B \cup C \cup D$  as in  $\sigma_1$ , receives no additional messages from  $E$ , and receives no message carrying or referring to  $y$ .

Therefore, since  $a$  executes  $x$  by time  $2\Delta$  in  $\sigma_1$ , determinism implies that  $a$  also executes  $x$  by time  $2\Delta$  in  $\sigma_2$ . Since  $a$  has identical timed histories in  $\sigma_1$  and  $\sigma_2$  through  $2\Delta$  and does not execute  $y$  in  $\sigma_1$  (no client submitted  $y$  in  $\sigma_1$ , so by external validity  $a$  cannot execute  $y$  in  $\sigma_1$ ),  $a$  could not have executed  $y$  by  $2\Delta$  in  $\sigma_2$ . In other words,

$$\sigma_2'' = \sigma_2' \cdot \mathbf{1}^C \cdot \mathbf{2}_{\sigma_1}^C \cdot \mathbf{2}_{\sigma_1}^A \cdot \mathbf{2}_{\sigma_1}^{B \cup D} \cdot \text{execute}_a(x) \in \mathcal{E}$$

The second indistinguishability holds because, in  $\sigma_3$ , all replicas receive the same first round messages from all replicas, replicas in  $A$  are Byzantine and silent after the first round, while in  $\sigma_2$  all messages after the first round from replicas in  $A$  to replicas outside  $A$  are delayed until after time  $R\Delta$ . Moreover, replicas in  $B \cup D$  receive commands  $x$  and  $y$  at respectively times  $\Delta$  and  $2\Delta$  in both executions, replicas in  $E$  receive  $y$  at exactly time  $\Delta$  in both executions, and Byzantine replicas in  $C$  send to replicas outside  $A$  exactly the messages that correct replicas in  $C$  send in  $\sigma_3$  through time  $R\Delta$ .

$$\sigma_2 = \sigma_2'' \cdot \mathbf{2}_{\sigma_5}^C \cdot \mathbf{2}_{\sigma_5}^E \cdots \mathbf{R}_{\sigma_3}^{B \cup C \cup D \cup E} \in \mathcal{E}$$

Since replica  $a$  executes  $x$  before  $y$ , replica  $b$  must execute  $x$  before  $y$ . Thus, replica  $b$  applies  $x$  in round  $R$ :  $\text{execute}_b(x) \in \mathbf{R}_{\sigma_3}^b$ .

**World 4 ( $\sigma_4$ ).** The construction is symmetric to  $\sigma_2$ . The Byzantine replicas in  $B$  equivocate as follows. Toward replicas in  $A$ , they behave exactly as the correct replicas in  $B$  behave in  $\sigma_5$  through time  $2\Delta$ ; in particular, they act as if they received  $y$  at exactly time  $\Delta$ . Toward replicas outside  $A$ , they behave exactly as the correct replicas in  $B$  behave in  $\sigma_3$  through time  $S\Delta$ ; in particular, they act as if they received  $x$  at exactly time  $\Delta$ . Below, we list the equations which lead to the fact that replica  $c$  executes  $y$  first in  $\sigma_3$ :

$$\begin{aligned} \sigma_4' &= \text{submit}(x) \cdot \text{submit}(y) \cdot \mathbf{1}^{A \cup C \cup E} \cdot \mathbf{1}^D \in \mathcal{E} \\ \sigma_4'' &= \sigma_4' \cdot \mathbf{1}^B \cdot \mathbf{2}_{\sigma_5}^B \cdot \mathbf{2}_{\sigma_5}^A \cdot \mathbf{2}_{\sigma_5}^{C \cup E} \cdot \text{execute}_a(y) \in \mathcal{E} \\ \sigma_4 &= \sigma_4'' \cdot \mathbf{2}_{\sigma_1}^B \cdot \mathbf{2}_{\sigma_1}^D \cdots \mathbf{S}_{\sigma_3}^{B \cup C \cup D \cup E} \in \mathcal{E} \end{aligned}$$

**Conclusion.** We have established that in  $\sigma_3$  replica  $b$  executes  $x$  before  $y$ , while replica  $c$  executes  $y$  before  $x$ . Since both replicas are correct,  $\sigma_3$  violates the Safety property of BFT SMR; contradiction. ◀

## 7 $n \geq 7f + 1$ Protocol

In this section we present a simpler, but suboptimal, generalized leaderless SMR protocol for the case  $n \geq 7f + 1$ . The benefit of this protocol is that the larger replication factor makes

the validation phase unnecessary. Thus, the  $n \geq 7f + 1$  protocol exposes a useful trade-off: it requires more replicas, but has a simpler and more efficient recovery path. In settings where recovery overhead is a throughput bottleneck, this simpler recovery procedure may lead to higher throughput per machine despite the larger replication factor [22].

The difficulty in the  $n \geq 5f + 1$  protocol is that recovery may only see  $n - 3f \geq 2f + 1$  matching FASTVOTE messages for the committed value. When  $n \geq 5f + 1$ , two quorums of size  $n - 3f$  may not intersect in a correct replica. Thus, two conflicting commands could each have enough FASTVOTE messages to be proposed during recovery, yet the replicas that voted for one command might contain no correct replica that also voted for the other. Thus, the  $n \geq 5f + 1$  protocol adds an additional validation phase to ensure that conflicting commands cannot miss each other during recovery.

For  $n \geq 7f + 1$ , this problem disappears. Any two sets of  $n - 3f$  replicas intersect in at least one correct replica. Therefore, if two conflicting commands each have  $n - 3f$  FASTVOTE messages, then some correct replica fast-voted for both commands. A correct replica includes every known conflicting command in the dependency set of any new command for which it sends a FASTVOTE message. Hence, whichever command the replica voted for second must include the first as a dependency. We can leverage this guarantee to simplify the recovery protocol and fully remove the validation phase.

The Status phase of the protocol thus changes in the following way. If the coordinator observes  $n - 3f$  matching prior VOTE messages, it repropose that value. Similarly, if it observes  $n - 3f$  matching FASTVOTE messages, it can directly repropose that value. Otherwise, if the fast votes disagree on the dependency set; the coordinator the command with the union of all reported dependencies. This is correct because the  $n - 3f$  quorums intersect with each other. All other phases of the protocol remain unchanged.

Algorithm 3 highlights the differences from the  $n \geq 5f + 1$  protocol.

■ **Algorithm 3** Modified Algorithm 2 for  $n \geq 7f + 1$

---

*All local state and handlers are unchanged from Algorithm 2, except that the status-phase handler is replaced by the following.*

---

```

1: upon receiving  $S \leftarrow n - f$  valid  $\langle \text{STATUS}, id, v, \cdot, \cdot \rangle$  and  $i = \text{COORDINATOR}(id, v)$  do
2:   if there exists  $n - 3f$   $\text{votes}[id]$  with  $val$  from  $S$  then
3:     send  $\langle \text{PROPOSE}, id, v, val, S, \perp, \perp \rangle$  to all  $\triangleright$  Possibly committed in an earlier view
4:   else if there exists  $n - 3f$   $\text{fastvotes}[id]$  with  $val = (x, \text{deps})$  from  $S$  then
5:     send  $\langle \text{VALIDATE}, id, v, (x, \text{deps}), S \rangle$  to all
6:     send  $\langle \text{PROPOSE}, id, v, val, S, \perp, \perp \rangle$  to all  $\triangleright$  Validation phase is unnecessary
7:   else
8:      $x \leftarrow$  unique non-equivocating command in the collected FASTVOTE messages
9:      $D \leftarrow$  union of all dependencies from  $fv[id]$ 
10:    send  $\langle \text{PROPOSE}, id, v, (x, D), S, \perp, \perp \rangle$  to all

```

---

*The validation-phase handlers from Algorithm 2 are removed. Furthermore, the view timer is set to  $4\Delta$  instead of  $6\Delta$ .*

---

## 8 Related Work

We compare leaderless Byzantine state-machine replication (SMR) with several related lines of work.



## 8.1 CFT Protocols

**Crash-fault-tolerant generalized SMR.** The EPaxos family of protocols satisfies a leaderless generalized SMR style of execution in the crash-fault-tolerant setting. EPaxos [32] and its successors, including Atlas [15], Caesar [6], EPaxos-TOQ [48], Accord [42], EPaxos\* [37], and BPaxos [51], allow replicas to propose commands independently and order commands by tracking dependencies between conflicting operations. These protocols are the closest conceptual predecessors to the abstraction studied in this paper. However, they rely heavily on the crash-fault model. Byzantine faults fundamentally complicate the dependency agreement problem, and crash-fault-tolerant techniques do not directly extend to the BFT setting. This paper studies the additional cost of obtaining EPaxos-style leaderless SMR in the presence of Byzantine behavior.

## 8.2 BFT Protocols

None of the work mentioned below achieve the optimal two-message-delay execution latency for every conflict-free command during synchronous periods while tolerating up to  $f$  failures.

**Leader-based BFT.** Classical BFT SMR protocols such as PBFT [11] and HotStuff [52] order commands through a leader. This design achieves the optimal Byzantine resilience bound of  $n = 3f + 1$ , but commands must pass through a leader-controlled agreement path before they can be executed. In PBFT-style protocols, a client first sends its request to the leader, after which the replicas run several communication phases to order and commit the command. As a result, even conflict-free commands take multiple message delays to execute.

PBFT [11] and other leader-based protocols like HotStuff-1 [24], and PoE [20] also support speculative execution. These protocols can execute early by allowing the system to roll back, repair, or later validate tentative execution results. Our execution latency property concerns final execution: when a correct replica executes a command, that execution is stable and is not later rolled back.

**Three message-delay fast path.** A large body of work reduces the optimistic latency of leader-based BFT protocols. Examples include FaB Paxos [3], Zyzzyva [26], SBFT [18], Kudzu [40], and Alpenglöw [25]. These protocols have a replication factor of  $n \geq 3f + 2p + 1$ , where  $p$  is the number of faulty replicas that can be tolerated on the fast path. They provide a fast path in which a proposal can be committed in three total message delays including the client's initial request. This is faster than traditional PBFT-style protocols, but the fast path remains leader-based.

**Two message delays + epsilon fast path.** Flutter/Blink [31] and Aspen [35] execute conflict-free commands without being sequenced through a leader, but they rely on tightly synchronized clocks. Their conflict-free latency under synchrony is  $2\Delta + \epsilon$ , where  $\epsilon$  accounts for clock discrepancies. In contrast, our protocol achieves latency  $2\Delta$  without requiring tightly synchronized clocks.

**Optimistic two message delay fast path.** Other protocols achieve low latency only under more restrictive failure assumptions. Wintermute [16] and Basil [47] have fast paths that can execute conflict-free commands in two message delays, but that fast path does not tolerate failures. Q/U [1] and Aliph [19] similarly not tolerate failures on its fast path. These protocols demonstrate that leaderless conflict-free execution is possible in favorable executions, but they do not provide the same failure-tolerant optimal execution latency guarantee required by our definition.

## 9 Conclusion

Leaderless BFT SMR has historically been less well understood than its crash-fault-tolerant counterpart. This paper narrows that gap by establishing tight bounds on the replication factor required for leaderless BFT SMR. We also illustrate a trade-off between replication cost and recovery efficiency by presenting a simpler protocol with a larger replication factor.

## References

- 1 Michael Abd-El-Malek, Gregory R. Ganger, Garth R. Goodson, Michael K. Reiter, and Jay J. Wylie. Fault-scalable byzantine fault-tolerant services. In *Proceedings of the Twentieth ACM Symposium on Operating Systems Principles*, SOSP '05, page 59–74, New York, NY, USA, 2005. Association for Computing Machinery.
- 2 Ittai Abraham, Dahlia Malkhi, and Alexander Spiegelman. Asymptotically optimal validated asynchronous byzantine agreement. In *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing*, PODC '19, page 337–346, New York, NY, USA, 2019. Association for Computing Machinery.
- 3 Lorenzo Alvisi and Jean-Philippe Martin. Fast Byzantine Consensus. *IEEE Transactions on Dependable and Secure Computing*, 3(03):202–215, July 2006.
- 4 Karolos Antoniadis, Julien Benhaim, Antoine Desjardins, Elias Poroma, Vincent Gramoli, Rachid Guerraoui, Gauthier Voron, and Igor Zablotchi. Leaderless consensus. *Journal of Parallel and Distributed Computing*, 176:95–113, 2023.
- 5 Apache Software Foundation. Apache cassandra, 2026. Version 5.0, accessed 2026-06-01.
- 6 Balaji Arun, Sebastiano Peluso, Roberto Palmieri, Giuliano Losa, and Binoy Ravindran. Speeding up consensus by chasing fast decisions. In *2017 47th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, pages 49–60, 2017.
- 7 Balaji Arun, Sebastiano Peluso, and Binoy Ravindran. ezbft: Decentralizing byzantine fault-tolerant state machine replication, 2019.
- 8 Kushal Babel, Andrey Chursin, George Danezis, Anastasios Kichidis, Lefteris Kokoris-Kogias, Arun Koshy, Alberto Sonnino, and Mingwei Tian. Mysticeti: Reaching the limits of latency with uncertified dags, 2025.
- 9 Rida Bazzi and Maurice Herlihy. Clairvoyant state machine replication, 2019.
- 10 Sam Blackshear, Andrey Chursin, George Danezis, Anastasios Kichidis, Lefteris Kokoris-Kogias, Xun Li, Mark Logan, Ashok Menon, Todd Nowacki, Alberto Sonnino, Brandon Williams, and Lu Zhang. Sui lutris: A blockchain combining broadcast and consensus. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, CCS '24, page 2606–2620, New York, NY, USA, 2024. Association for Computing Machinery.
- 11 Miguel Castro and Barbara Liskov. Practical byzantine fault tolerance. In *Proceedings of the Third Symposium on Operating Systems Design and Implementation*, OSDI '99, page 173–186, USA, 1999. USENIX Association.
- 12 Tyler Crain, Vincent Gramoli, Mikel Larrea, and Michel Raynal. Dbft: Efficient leaderless byzantine consensus and its application to blockchains. In *2018 IEEE 17th International Symposium on Network Computing and Applications (NCA)*, pages 1–8, 2018.
- 13 George Danezis, Lefteris Kokoris-Kogias, Alberto Sonnino, and Alexander Spiegelman. Narwhal and tusk: a dag-based mempool and efficient bft consensus. In *Proceedings of the Seventeenth European Conference on Computer Systems*, EuroSys '22, page 34–50, New York, NY, USA, 2022. Association for Computing Machinery.
- 14 Michael Eischer and Tobias Distler. Egalitarian byzantine fault tolerance. In *2021 IEEE 26th Pacific Rim International Symposium on Dependable Computing (PRDC)*, pages 1–10, 2021.
- 15 Vitor Enes, Carlos Baquero, Tuanir França Rezende, Alexey Gotsman, Matthieu Perrin, and Pierre Sutra. State-machine replication for planet-scale systems. In *Proceedings of the*

- Fifteenth European Conference on Computer Systems, EuroSys '20*, New York, NY, USA, 2020. Association for Computing Machinery.
- 16 Tuanir Franca Rezende. *Leaderless state-machine replication : from fail-stop to Byzantine failures*. Theses, Institut Polytechnique de Paris, December 2021.
  - 17 Neil Giridharan, Florian Suri-Payer, Ittai Abraham, Lorenzo Alvisi, and Natacha Crooks. Autobahn: Seamless high speed bft. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles, SOSP '24*, page 1–23, New York, NY, USA, 2024. Association for Computing Machinery.
  - 18 Guy Golan Gueta, Ittai Abraham, Shelly Grossman, Dahlia Malkhi, Benny Pinkas, Michael Reiter, Dragos-Adrian Seredinschi, Orr Tamir, and Alin Tomescu. Sbft: A scalable and decentralized trust infrastructure. In *2019 49th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, pages 568–580, 2019.
  - 19 Rachid Guerraoui, Nikola Knežević, Vivien Quéma, and Marko Vukolić. The next 700 bft protocols. In *Proceedings of the 5th European Conference on Computer Systems, EuroSys '10*, page 363–376, New York, NY, USA, 2010. Association for Computing Machinery.
  - 20 Suyash Gupta, Jelle Hellings, Sajjad Rahnama, and Mohammad Sadoghi. Proof-of-execution: Reaching consensus through fault-tolerant speculation, 2021.
  - 21 Suyash Gupta, Jelle Hellings, and Mohammad Sadoghi. Rcc: Resilient concurrent consensus for high-throughput secure transaction processing. In *2021 IEEE 37th International Conference on Data Engineering (ICDE)*, pages 1392–1403, 2021.
  - 22 Suyash Gupta, Sajjad Rahnama, Shubham Pandey, Natacha Crooks, and Mohammad Sadoghi. Dissecting bft consensus: In trusted components we trust! In *Proceedings of the Eighteenth European Conference on Computer Systems, EuroSys '23*, page 521–539, New York, NY, USA, 2023. Association for Computing Machinery.
  - 23 Maurice P. Herlihy and Jeannette M. Wing. Linearizability: a correctness condition for concurrent objects. *ACM Trans. Program. Lang. Syst.*, 12(3):463–492, July 1990.
  - 24 Dakai Kang, Suyash Gupta, Dahlia Malkhi, and Mohammad Sadoghi. Hotstuff-1: Linear consensus with one-phase speculation, 2025.
  - 25 Quentin Kniep, Jakub Sliwinski, and Roger Wattenhofer. Solana alpenglows consensus: Increased bandwidth, reduced latency. White paper, Anza Research / ETH Zurich, 2025.
  - 26 Ramakrishna Kotla, Lorenzo Alvisi, Mike Dahlin, Allen Clement, and Edmund Wong. Zyzzyva: Speculative byzantine fault tolerance. *ACM Trans. Comput. Syst.*, 27(4), January 2010.
  - 27 Petr Kuznetsov, Andrei Tonkikh, and Yan X Zhang. Revisiting optimal resilience of fast byzantine consensus. In *Proceedings of the 2021 ACM Symposium on Principles of Distributed Computing, PODC'21*, page 343–353, New York, NY, USA, 2021. Association for Computing Machinery.
  - 28 Leslie Lamport. Generalized consensus and paxos. Technical Report MSR-TR-2005-33, March 2005.
  - 29 Andrew Miller, Yu Xia, Kyle Croman, Elaine Shi, and Dawn Song. The honey badger of bft protocols. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, CCS '16*, page 31–42, New York, NY, USA, 2016. Association for Computing Machinery.
  - 30 Zarko Milosevic, Martin Biely, and André Schiper. Bounded delay in byzantine-tolerant state machine replication. In *2013 IEEE 32nd International Symposium on Reliable Distributed Systems*, pages 61–70, 2013.
  - 31 Matteo Monti, Martina Camaioni, and Pierre-Louis Roman. Fast leaderless byzantine total order broadcast. *arXiv preprint arXiv:2412.14061*, 2024.
  - 32 Iulian Moraru, David G. Andersen, and Michael Kaminsky. There is more consensus in egalitarian parliaments. In *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles, SOSP '13*, page 358–372, New York, NY, USA, 2013. Association for Computing Machinery.

- 33 Diego Ongaro and John Ousterhout. In search of an understandable consensus algorithm. In *Proceedings of the 2014 USENIX Conference on USENIX Annual Technical Conference*, USENIX ATC'14, page 305–320, USA, 2014. USENIX Association.
- 34 Miguel Pires, Srivatsan Ravi, and Rodrigo Rodrigues. Generalized paxos made byzantine (and less complex), 2017.
- 35 Daniel Qian, Xiyu Hao, Jinkun Geng, Yuncheng Yao, Aurojit Panda, Jinyang Li, and Anirudh Sivaraman. Revisiting speculative leaderless protocols for low-latency bft replication, 2026.
- 36 Tuanir França Rezende and Pierre Sutra. Leaderless State-Machine Replication: Specification, Properties, Limits. In Hagit Attiya, editor, *34th International Symposium on Distributed Computing (DISC 2020)*, volume 179 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 24:1–24:17, Dagstuhl, Germany, 2020. Schloss Dagstuhl–Leibniz-Zentrum für Informatik.
- 37 Fedor Ryabinin, Alexey Gotsman, and Pierre Sutra. Making Democracy Work: Fixing and Simplifying Egalitarian Paxos. In Andrei Arusoaie, Emanuel Onica, Michael Spear, and Sara Tucci-Piergiovanni, editors, *29th International Conference on Principles of Distributed Systems (OPODIS 2025)*, volume 361 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 22:1–22:19, Dagstuhl, Germany, 2026. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.
- 38 Fedor Ryabinin, Alexey Gotsman, and Pierre Sutra. Revisiting lower bounds for two-step consensus, 2026.
- 39 Fred B. Schneider. Implementing fault-tolerant services using the state machine approach: A tutorial. *ACM Computing Surveys*, 22(4):299–319, December 1990.
- 40 Victor Shoup, Jakub Sliwinski, and Yann Vonlanthen. Kudzu: Fast and Simple High-Throughput BFT. In Dariusz R. Kowalski, editor, *39th International Symposium on Distributed Computing (DISC 2025)*, volume 356 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 42:1–42:19, Dagstuhl, Germany, 2025. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.
- 41 Nibesh Shrestha and Mohan Kumar. Revisiting ezbft: A decentralized byzantine fault tolerant protocol with speculation, 2019.
- 42 Benedict Elliott Smith, Tony Zhang, Blake Eggleston, and Scott Andreas. Cep-15: Fast general purpose transactions. Apache Cassandra Enhancement Proposal (CEP-15) Whitepaper, 2021.
- 43 Yee Jiun Song and Robbert Renesse. Bosco: One-step byzantine asynchronous consensus. In *Proceedings of the 22nd International Symposium on Distributed Computing*, DISC '08, page 438–450, Berlin, Heidelberg, 2008. Springer-Verlag.
- 44 Alexander Spiegelman, Neil Giridharan, Alberto Sonnino, and Lefteris Kokoris-Kogias. Bullshark: Dag bft protocols made practical. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, CCS '22*, page 2705–2718, New York, NY, USA, 2022. Association for Computing Machinery.
- 45 Chrysoula Stathakopoulou, Tudor David, Matej Pavlovic, and Marko Vukolić. Mir-bft: High-throughput robust bft for decentralized networks, 2021.
- 46 Chrysoula Stathakopoulou, Matej Pavlovic, and Marko Vukolić. State machine replication scalability made simple. In *Proceedings of the Seventeenth European Conference on Computer Systems*, EuroSys '22, page 17–33, New York, NY, USA, 2022. Association for Computing Machinery.
- 47 Florian Suri-Payer, Matthew Burke, Zheng Wang, Yunhao Zhang, Lorenzo Alvisi, and Natacha Crooks. Basil: Breaking up bft with acid (transactions). In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles, SOSP '21*, page 1–17, New York, NY, USA, 2021. Association for Computing Machinery.
- 48 Sarah Tollman, Seo Jin Park, and John Ousterhout. EPaxos revisited. In *18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21)*, pages 613–632. USENIX Association, April 2021.

- 49 Robbert Van Renesse and Deniz Altinbuken. Paxos made moderately complex. *ACM Comput. Surv.*, 47(3), February 2015.
- 50 Michael Whittaker, Neil Girdharan, Adriana Szekeres, Joseph M. Hellerstein, and Ion Stoica. Bipartisan paxos: A modular state machine replication protocol, 2020.
- 51 Michael J. Whittaker, Neil Girdharan, Adriana Szekeres, Joseph M. Hellerstein, and Ion Stoica. Sok: A generalized multi-leader state machine replication tutorial. *J. Syst. Res.*, 1(1), 2021.
- 52 Maofan Yin, Dahlia Malkhi, Michael K. Reiter, Guy Golan Gueta, and Ittai Abraham. Hotstuff: Bft consensus with linearity and responsiveness. In *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing*, PODC '19, page 347–356, New York, NY, USA, 2019. Association for Computing Machinery.

## **A Additional Related Work**

### **A.1 CFT Protocols**

**Generalized consensus.** Generalized consensus protocols, beginning with Generalized Paxos [28] and later variants [34], also exploit the fact that non-conflicting commands need not be totally ordered. These protocols are conceptually close to generalized leaderless SMR. However, when conflicts are detected, they typically require a new ballot, recovery path, or additional coordination step. This can delay even conflict-free commands that are entangled with unresolved ordering information. As a result, these protocols do not directly provide the leaderless execution latency property considered here, where conflict-free commands must execute within two total message delays even in the presence of up to  $f$  Byzantine failures.

### **A.2 BFT Protocols**

**Single-shot agreement.** Archipelago [4] is leaderless consensus protocol in the sense that the consensus protocol can terminate without relying on one designated replica to drive progress. Our use of the term is different. We study multi-shot SMR, where clients continuously submit commands, replicas must execute a growing history, and the protocol should avoid totally ordering commands that do not conflict. Bosco [43] studies single-shot Byzantine agreement with a one-message-delay fast decision path. Unfortunately, as discussed in section 4, using a single-shot agreement protocol with a fast path in a black-box manner does not immediately satisfy Invariant 2 (Visibility).

**No fast path.** Asynchronous consensus protocols [29, 2] and DAG protocols [13, 44] have multiple proposers, but do not have optimal execution latency when there are conflict-free commands. Byblos [9] avoids ordering non-conflicting commands, but its latency is always at least five message delays. Similarly, Egalitarian BFT [14] does not have a fast path. DBFT [12] reduces consensus to binary agreement instances that do not rely on a single leader, but it does not have a fast path that for conflict-free commands.

**Restricted two message delay fast path.** Protocols such as Sui Lutriss [10], Mysticeti [8] have a fast path with two message delays, but this applies only to single-writer operations. In contrast, the leaderless protocols studied in this paper are not limited to these special cases. They support two message-delay execution for any conflict-free commands.

**Log sharding.** Multi-leader BFT protocols such as Mir-BFT [45], ISS [46], BFT Mencius [30], and RCC [21] improve throughput by assigning different leaders to different log positions or partitions of the log. This reduces the bottleneck of a single leader, but the abstraction remains fundamentally log-based. Commands are assigned to slots, and execution generally proceeds according to the order of those slots. Thus, even if two commands are

conflict-free, a command assigned to a later slot may have to wait for earlier slots to commit before it can be executed. These protocols therefore do not satisfy generalized leaderless SMR as defined in this paper, since conflict-free commands are still delayed by unrelated earlier log positions.